



COMP 4300

Intro to Game Programming

Lecture 19

Advanced Game Main Loops

Game Loops

- Material for these slides is available in textbook: Game Programming Patterns
- <http://gameprogrammingpatterns.com/game-loop.html>

In the Beginning...

- You may recall simpler console programs you wrote while learning how to program
- Programs had the following structure:
 1. Run some initial code
 2. Print instruction for user to the screen
 3. Wait for input from the user
 4. Store the user input somehow
 5. Continue with program code

Simple Program Structure

```
1. int main()
2. {
3.     float input = 0;
4.     while (true)
5.     {
6.         cout << "Enter Distance in Feet\n";
7.         cin  >> input;
8.         cout << "That is " << (input*12) << " inches\n";
9.     }
10. }
```

Simple Program Structure

```
1.    // display some instructions
2.    while (true)
3.    {
4.        // wait for user input (blocking)
5.        // do something interesting with input
6.        // display more instructions (or quit)
7.    }
```

Graphical User Interfaces

- Even more modern GUIs follow the same basic structure, since nothing in the GUI changes unless user inputs values
- Example: Word Processor
 - Display current document
 - Wait for user to type or click
 - Handle the user input somehow
 - Repeat

Graphical User Interfaces

```
1.    // display initial GUI screen
2.    while (true)
3.    {
4.        // poll for user input events
5.        // update internal variables based on input
6.        // update user interface
7.    }
```

Game Main Loops

- Unlike many other programs, games **keep moving** when the user doesn't input
- The game won't be paused just because the user isn't doing anything
- Clock keeps ticking, NPCs keep moving, Animations keep animating
- Key: We don't WAIT for user input

Game Loops

```
1.    // display initial GUI screen
2.    while (true)
3.    {
4.        // process user input if any exists
5.        // update internal game simulation by one step
6.        // render current game screen
7.    }
```

Game Loops

```
1.    // display initial GUI screen
2.    while (true)
3.    {
4.        processInput();
5.        update();
6.        render();
7.    }
```

A4 Game Engine Example

```
1. void update()  
2. {  
3.     sUserInput();           // processInput()  
4.     m_entityManager.update(); // update()  
5.     sAI();  
6.     sMovement();  
7.     sLifespan();  
8.     sCollision();  
9.     sAnimation();          // render()  
10.    sRender();  
11. }
```

Game Loop Speed

- When programs use blocking input, they can run the main loop **at any speed** since it's limited by user input speed
- With non-blocking input in a game loop, we now need to decide **how fast** the loop spins
- Each update() advances the game world by the **same amount each call** (example: speed 5, $x += 5$)
- Game world clock 'ticks' forward by 1
- Player's real-world clock ticks at a **different** speed
- How many game loop cycles in one second determines the game's '**frames per second**' count

Frames Per Second

```
1.    // display initial GUI screen
2.    while (true)
3.    {
4.        processInput();
5.        update();
6.        render();
7.    }
    // 60 loops per second = 60 render() calls = 60 FPS
```

Frames Per Second

- Graphically, more FPS = **smoother** play
- FPS is typically determined by two things:
 - How much work done on each loop
 - How fast is the underlying hardware
- Older games had the advantage of knowing the **exact hardware** it would run on, so they knew how 'fast' the game would run
- Modern computer games must run on a **variety of hardware**, so we must accommodate for that

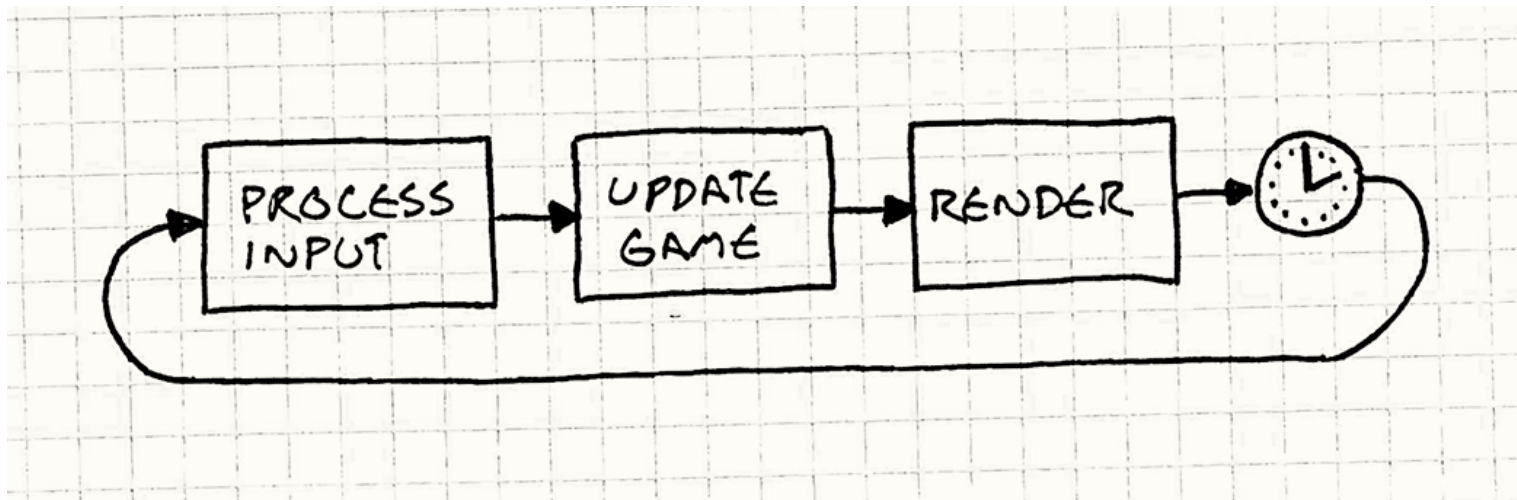
Game Loops / Hardware

- The flexibility you have with your game loop can depend on many factors
- OS: How often can you poll events?
- OS: Are hardware input events blocking?
- Monitor: Can't render more often than monitor refresh rate (Vsync)
- Networking: Type of net traffic can determine time outs, packet loss, etc (not covered here)

Setting Fixed FPS Count

- Let's say we want to **fix game** to 60fps
- Note: we have cheated so far by letting SFML do this for us with built-in command:
 - `m_window.setFramerateLimit(60);`
- To do it manually, we must **limit** our game loop to run at most every $1000/60 \sim 16\text{ms}$
- We will add a **delay** to the end of each loop

Setting Fixed FPS



Setting Fixed FPS

```
1. float MS_PER_FRAME = 1000.0 / 60.0;
2. while (true)
3. {
4.     float start = getCurrentTimeMS();
5.     processInput();
6.     update();
7.     render();
8.     float elapsed = getCurrentTimeMS() - start;
9.     sleep(max(0, MS_PER_FRAME - elapsed));
10. }
```

Setting Fixed FPS

- Adding the `sleep()` at the end of the loop ensures we don't update **too quickly**
- Unfortunately, there's **no way** to ensure that slow code doesn't make your game run too slowly
- In this case, must cut computation time to ensure the FPS target can be reached

Potential Problem

```
1. float MS_PER_FRAME = 1000.0 / 60.0;
2. while (true)
3. {
4.     float start = getCurrentTimeMS();
5.     processInput();
6.     update();
7.     render();
8.     float elapsed = getCurrentTimeMS() - start;
9.     sleep(max(0, MS_PER_FRAME - elapsed));
10. }
```

Potential Problem

```
1. float MS_PER_FRAME = 1000.0 / 60.0;
2. while (true)
3. {
4.     float start = getCurrentTimeMS();
5.     processInput();
6.     update(); // this could be too slow sometimes
7.     render();
8.     float elapsed = getCurrentTimeMS() - start;
9.     sleep(max(0, MS_PER_FRAME - elapsed));
10. }
```

Potential Problem

```
1. float MS_PER_FRAME = 1000.0 / 60.0;
2. while (true)
3. {
4.     float start = getCurrentTimeMS();
5.     processInput();
6.     update();
7.     render(); // this could be too slow sometimes
8.     float elapsed = getCurrentTimeMS() - start;
9.     sleep(max(0, MS_PER_FRAME - elapsed));
10. }
```

Note: SFML

- `Sf::Window::setFrameLimit(limit)`
- Limits the framerate to a maximum
- If a limit is set, the window will use a small delay after each call to `display()` to ensure that the current frame lasted long enough to match the framerate limit. SFML will try to match the given limit as much as it can, but since it internally uses `sf::sleep`, whose precision depends on the underlying OS, the results may be a little unprecise as well (for example, you can get 65 FPS when requesting 60).

Quick Notes

- The following slides will show how to implement 'catching up' after lag frames
- In some cases we want to do this, and in other cases we actually don't want to
- These slides are to show you **one way** to implement different techniques in your game engine if this is your goal

Game Simulation Scaling

- The problem we face is as follows:
 1. Each update advances game time by some amount
 2. It takes some amount of **real** time to compute that
- If 2 takes longer than 1: the game slows down
- Example: If it takes longer than 16ms to update the game world by 16ms equivalent, we can't keep up
- If we could advance the game world by more than 16ms, we can update less frequently and keep up to date
- Idea: introduce a variable that we will use to update the game world simulation (**variable time step**)

Variable Time Step Loop

```
1. float lastTime = getCurrentTimeMS();
2. while (true)
3. {
4.     float current = getCurrentTimeMS();
5.     float elapsed = current - lastTime;
6.     processInput();
7.     update(elapsed); // variable update time
8.     render();
9.     lastTime = current;
10. }
```

Variable Time Step

- Each frame we calculate how much **real time has passed** since the last game update
- When we update the game state, we use the real elapsed time as a **scale** variable
- Example: Bullet flying through the air
 - Each time step update it with velocity
 - Variable time: scale velocity with elapsed time

Variable Time Step

- Advantages
 - Game plays at **consistent** rate on diff hardware
 - Players with faster machines get **smoother** FPS
- Disadvantages
 - Inconsistent effects based on variable FPS
 - Floating point errors produce diff results
 - Network desynchronization
 - Physics simulations yield diff results (collisions)

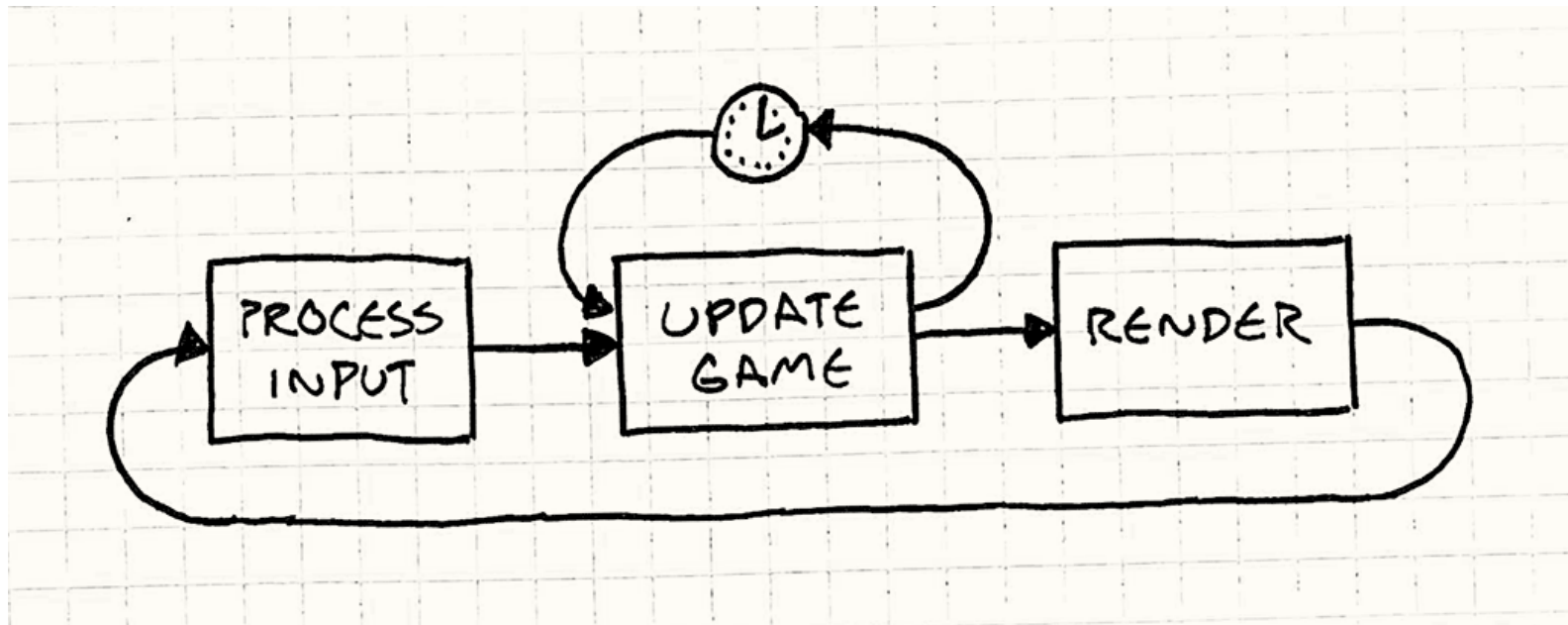
Variable Time Problems

```
1. float lastTime = getCurrentTimeMS();
2. while (true)
3. {
4.     float current = getCurrentTimeMS();
5.     float elapsed = current - lastTime;
6.     processInput();
7.     update(elapsed); // could be floating point error
8.     render();
9.     lastTime = current;
10. }
```

How Rendering Fits

- Whether or not we use variable time steps, rendering is **not affected**
- Rendering essentially just takes the **current state** of the game and draws it
- So instead of variably updating the game simulation, let's render **at variable times**
- This lets us go back fixed game time steps

Variable Number of Fixed Updates



Real-World Analogy

- You're tasked to make a cheeseburger **every minute** at a fast-food place
- It takes you about 15 seconds to make a cheeseburger, so you have 45 sec **rest** / min
- One burger slips and falls, and it takes you 2 minutes to clean it up. You're now **behind** by 2
- Since you can make 4 burgers per minute, you **catch up** next minute by making 3 total

Burger Analogy

Desired: 1 Burger Per Minute



60 sec



120s Accident



15s

45s



```
1. float previous = getCurrentTimeMS();
2. float lag = 0.0f;
3. while (true)
4. {
5.     float current = getCurrentTimeMS();
6.     float elapsed = current - previous;
7.     previous = current
8.     lag += elapsed;
9.     processInput();
10.    while (lag >= MS_PER_UPDATE)
11.    {
12.        update();
13.        lag -= MS_PER_UPDATE
14.    }
15.    render();
16. }
```

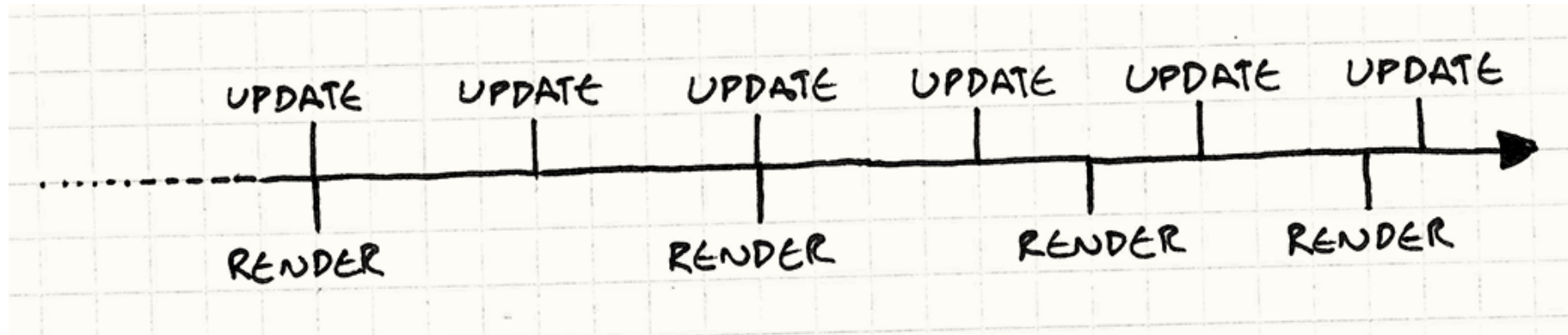
What does this do?

- Lag measures 'how far behind' the game's clock is behind the real world clock (ex: if one update or render took too long)
- Inner loop updates the game to 'catch up' until all of the lag is gone, with fixed step updates (which are consistent)
- Once we catch up, we render the scene
- Game simulates at a constant rate using fix time steps in a 'safe' way + 'catch up' for lagged frames
- Rendering may appear to 'skip' while the game catches up, but at least the simulation speed is fixed on diff hardware
- Note: We can only 'catch up' if 'lag frames' are infrequent

Rendering Decoupling

- Important: We have now **removed** the rendering **synchronization** from `update()`
- No longer call one render per update
- This means we could put rendering into another **thread** (for example)
- But some issues can arise from this, namely: rendering **between** updates

Rendering Between Updates



Rendering / Interpolation

- If we render between updates, game world could be the **same** for 2 frames, which looks quite choppy / bad
- We can **interpolate** / do basic movement calculations inside rendering engine
 - `render(lag / MS_PER_UPDATE)`
- Physics may not be exact for this render, but the game world will be exact in future

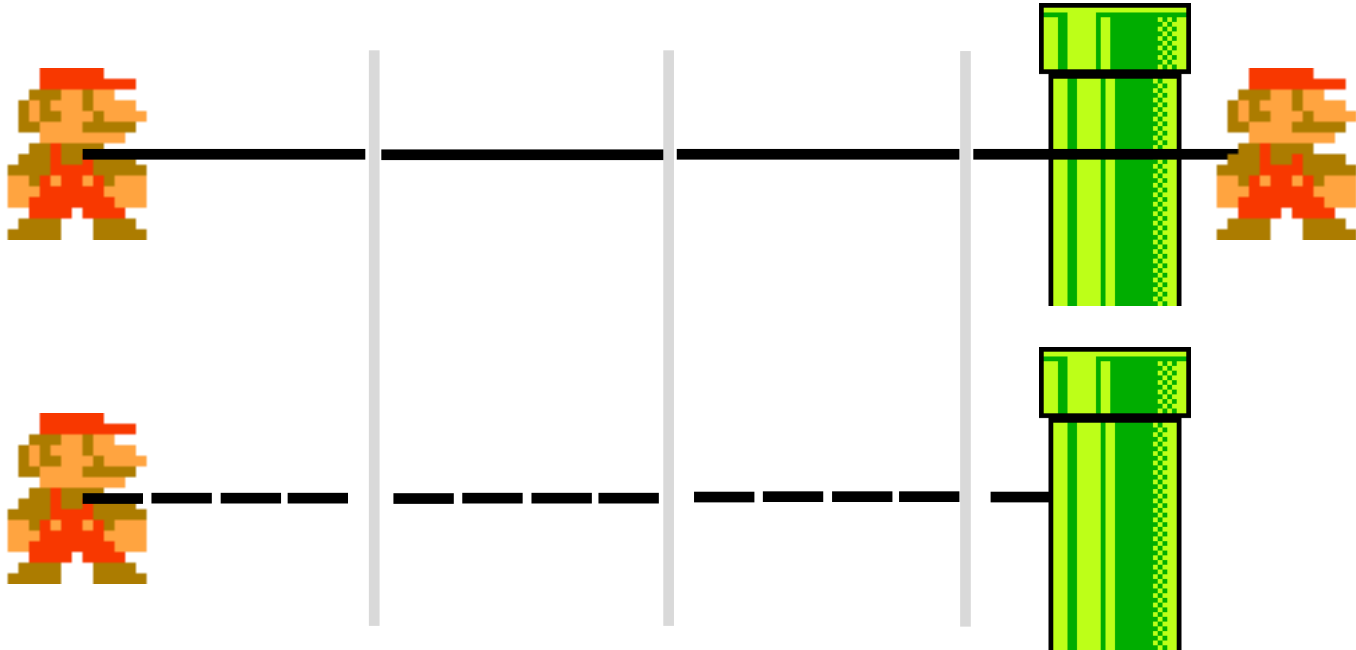
Game Speed + Collision Issues

- Given our basic way of doing collision detection, an entity going fast enough could **pass through** a game tile / bullet
- We 'fixed' this in assignment 3 by setting a **maximum speed** for entities
- As long as the maximum speed is **less than the thickness** of smallest tile, works
- But what if we want to allow entities in our game to go at **faster** speeds than this allows?

Multiple Update Loops

- One simple solution is to increase the number of game ticks per second
- Example:
 - We want max player speed 100 pixels / frame
 - Tile size is fixed 32 pixels
 - Instead: fix player speed to 25, and update game world 4 times before rendering
- Trade CPU effort for simulation accuracy

Multiple Update Loops



Mario 64 Examples

- Coin Falling Through Floor
 - <https://www.youtube.com/watch?v=oQ7dXvMfr30>
- Walls, Floors, & Ceilings
 - <https://www.youtube.com/watch?v=UnU7DJXiMAQ>
- Parallel Universes
 - <https://www.youtube.com/watch?v=kpk2tdsPh0A>